

1. Practical1

```
#include<stdio.h>
#include<unistd.h>
#include<stdlib.h>
#include<sys/wait.h>
int main(){
char command[100];
printf("Enter a linux command: ");
scanf("%s",command);
int pid;
pid = fork();
if(pid==0){
printf("Child PID: %d\n",getpid());
execlp(command,command,NULL);
}
else{
printf("Parent PID: %d\n",getpid());
wait(NULL);
}
return 0;
}
```

Output:

```
karamsettsaicharan@SaiCharan:~/Practicals$ ./practical1
Enter a linux command: ls
Parent PID: 1074
Child PID: 1075
copy.txt practical1 practical2 practical3 practical4 practical5 practical5_1 priority roundrobin sample.txt sjf.c
fcfs.c practical1.c practical2.c practical3.c practical4.c practical5.c practical5_1.c priority.c roundrobin.c sjf
```

2. Practical2

```
GNU nano 2.7.1
#include<stdio.h>
#include<unistd.h>
#include<fcntl.h>
int main(){
int fd1, fd2;
char message[100];
int n;
fd1 = open("sample.txt", O_RDONLY|O_CREAT, 0666);
fd2 = open("copy.txt", O_WRONLY|O_CREAT, 0666);
n = read(fd1, message, 100);
write(fd2, message, n);
close(fd1);
close(fd2);
printf("File copied Successfully\n");
return 0;
}
```

Output:

```
karamsettisaicharan@SaiCharan:~/Practicals$ ./practical2
File copied Successfully
karamsettisaicharan@SaiCharan:~/Practicals$ cat sample.txt
Hello World
karamsettisaicharan@SaiCharan:~/Practicals$ cat copy.txt
Hello World
karamsettisaicharan@SaiCharan:~/Practicals$ |
```

ii) strace cat sample.txt

```
karamsettisaicharan@SaiCharan:~/Practicals$ strace cat sample.txt
execve("/usr/bin/cat", ["cat", "sample.txt"], 0x7ffce65feb58 /* 26 vars */)
= 0
brk(NULL)                                = 0x6448e3793000
mmap(NULL, 8192, PROT_READ|PROT_WRITE, MAP_PRIVATE|MAP_ANONYMOUS, -1, 0) = 0x777020cf6000
access("/etc/ld.so.preload", R_OK)       = -1 ENOENT (No such file or directory)
openat(AT_FDCWD, "/etc/ld.so.cache", O_RDONLY|O_CLOEXEC) = 3
fstat(3, {st_mode=S_IFREG|0644, st_size=20085, ...}) = 0
mmap(NULL, 20085, PROT_READ, MAP_PRIVATE, 3, 0) = 0x777020cf1000
close(3)                                 = 0
openat(AT_FDCWD, "/usr/lib/x86_64-linux-gnu/libselinux.so.1", O_RDONLY|O_CLOEXEC) = 3
read(3, "\177ELF\2\1\1\0\0\0\0\0\0\0\0\0\3\0>\0\1\0\0\0\0\0\0\0\0\0"..., 832) = 832
fstat(3, {st_mode=S_IFREG|0644, st_size=199120, ...}) = 0
mmap(NULL, 206288, PROT_READ, MAP_PRIVATE|MAP_DENYWRITE, 3, 0) = 0x777020cbe000
mmap(0x777020cc5000, 135168, PROT_READ|PROT_EXEC, MAP_PRIVATE|MAP_FIXED|MAP_DENYWRITE, 3, 0x7000) = 0x777020cc5000
mmap(0x777020ce6000, 28672, PROT_READ, MAP_PRIVATE|MAP_FIXED|MAP_DENYWRITE, 3, 0x28000) = 0x777020ce6000
mmap(0x777020ced000, 8192, PROT_READ|PROT_WRITE, MAP_PRIVATE|MAP_FIXED|MAP_DENYWRITE, 3, 0x2f000) = 0x777020ced000
mmap(0x777020cef000, 5584, PROT_READ|PROT_WRITE, MAP_PRIVATE|MAP_FIXED|MAP_ANONYMOUS, -1, 0) = 0x777020cef000
close(3)                                 = 0
openat(AT_FDCWD, "/usr/lib/x86_64-linux-gnu/libgcc_s.so.1", O_RDONLY|O_CLOEXEC) = 3
read(3, "\177ELF\2\1\1\0\0\0\0\0\0\0\0\0\3\0>\0\1\0\0\0\0\0\0\0\0\0"..., 832) = 832
fstat(3, {st_mode=S_IFREG|0644, st_size=187120, ...}) = 0
mmap(NULL, 185256, PROT_READ, MAP_PRIVATE|MAP_DENYWRITE, 3, 0) = 0x777020c90000
mmap(0x777020c94000, 147456, PROT_READ|PROT_EXEC, MAP_PRIVATE|MAP_FIXED|MAP_DENYWRITE, 3, 0x4000) = 0x777020c94000
mmap(0x777020cb8000, 16384, PROT_READ, MAP_PRIVATE|MAP_FIXED|MAP_DENYWRITE, 3, 0x28000) = 0x777020cb8000
mmap(0x777020cbc000, 8192, PROT_READ|PROT_WRITE, MAP_PRIVATE|MAP_FIXED|MAP_D
```

3. Practical3

```
GNU nano 2.7.1
#include<stdio.h>
#include<stdlib.h>
#include<sys/wait.h>
#include<unistd.h>
int main(){
int pid = fork();
if(pid<0){
printf("Fork Failed\n");
}
else if (pid==0){
printf("Child PID: %d\n",getpid());
printf("Parent PID: %d\n",getppid());
sleep(10);
}
else{
printf("Parent PID: %d\n",getpid());
sleep(10);
}
return 0;
}
```

Output:

```
karamsethisaicharan@SaiCharan:~/Practicals$ ./practical3
Parent PID: 1259
Child PID: 1260
Parent PID: 1259
```

4. Practical4

```
#include <stdio.h>
#include <unistd.h>
#include <sys/wait.h>
int main()
{
    int p1, p2, p3;
    p1 = fork();
    if (p1 == 0)
    {
        printf("Child 1 PID: %d\n", getpid());
        return 0;
    }
    p2 = fork();
    if (p2 == 0)
    {
        printf("Child 2 PID: %d\n", getpid());
        return 0;
    }
    p3 = fork();
    if (p3 == 0)
    {
        printf("Child 3 PID: %d\n", getpid());
        return 0;
    }
    waitpid(p2, NULL, 0);
    printf("Parent: Child 2 completed\n");
    return 0;
}
```

Output:

```
karamsettsaicharan@SaiCharan:~/Practicals$ ./practical4
Child 1 PID: 1293
Child 2 PID: 1294
Child 3 PID: 1295
Parent: Child 2 completed
karamsettsaicharan@SaiCharan:~/Practicals$ |
```

ii)zombieprocess.c

```
#include<stdio.h>
#include<unistd.h>
#include<stdlib.h>
#include<sys/wait.h>
int main(){
if(fork()==0){
printf("Child process\n");
printf("CP PID is=%d\n",getpid());
printf("CP execution is done\n");
exit(0);
}else{
printf("Parent process\n");
printf("pp pid is = %d\n",getpid());
sleep(10);
printf("PP done");
}
return 0;
}
```

```
karamsettisaicharan@SaiCharan:~$ ./zombie
Parent process
Child process
pp pid is = 1261
CP PID is=1262
CP execution is done
```

5. Practical5 (filter commands)

```
#include<stdio.h>
#include<unistd.h>
#include<sys/wait.h>
int main(){
    int fd[2];
    pid_t p1,p2;
    pipe(fd);
    p1=fork();
    if(p1==0){
        close(fd[0]);
        dup2(fd[1],STDOUT_FILENO);
        close(fd[1]);
        execlp("ls","ls","-l",NULL);
    }
    p2 = fork();
    if(p2==0){
        close(fd[1]);
        dup2(fd[0],STDIN_FILENO);
        close(fd[0]);
        execlp("grep","grep",".c",NULL);
    }
    close(fd[0]);
    close(fd[1]);
    wait(NULL);
    wait(NULL);
    return 0;
}
```

Output:

```
karamsettisaicharan@SaiCharan:~/Practicals$ ./practical5_1
-rw-r--r-- 1 karamsettisaicharan karamsettisaicharan 12 Sep 9 09:28 copy.txt
-rw-r--r-- 1 karamsettisaicharan karamsettisaicharan 118 Sep 1 05:33 fcfs.c
-rwxr-xr-x 1 karamsettisaicharan karamsettisaicharan 16232 Aug 24 13:23 practical1
-rw-r--r-- 1 karamsettisaicharan karamsettisaicharan 336 Sep 9 09:25 practical1.c
-rwxr-xr-x 1 karamsettisaicharan karamsettisaicharan 16176 Sep 9 09:00 practical2
-rw-r--r-- 1 karamsettisaicharan karamsettisaicharan 324 Sep 9 09:27 practical2.c
-rwxr-xr-x 1 karamsettisaicharan karamsettisaicharan 16128 Aug 24 13:36 practical3
-rw-r--r-- 1 karamsettisaicharan karamsettisaicharan 318 Sep 9 09:29 practical3.c
-rwxr-xr-x 1 karamsettisaicharan karamsettisaicharan 16128 Aug 24 13:54 practical4
-rw-r--r-- 1 karamsettisaicharan karamsettisaicharan 500 Sep 9 09:30 practical4.c
-rwxr-xr-x 1 karamsettisaicharan karamsettisaicharan 16216 Sep 9 09:20 practical5
-rw-r--r-- 1 karamsettisaicharan karamsettisaicharan 409 Sep 9 09:21 practical5.c
-rwxr-xr-x 1 karamsettisaicharan karamsettisaicharan 16216 Sep 9 09:15 practical5_1
-rw-r--r-- 1 karamsettisaicharan karamsettisaicharan 389 Sep 9 09:32 practical5_1.c
-rwxr-xr-x 1 karamsettisaicharan karamsettisaicharan 16096 Sep 1 05:54 priority
-rw-r--r-- 1 karamsettisaicharan karamsettisaicharan 2599 Sep 6 09:16 priority.c
-rwxr-xr-x 1 karamsettisaicharan karamsettisaicharan 16104 Sep 1 04:04 roundrobin
-rw-r--r-- 1 karamsettisaicharan karamsettisaicharan 759 Sep 7 03:21 roundrobin.c
-rw-r--r-- 1 karamsettisaicharan karamsettisaicharan 12 Sep 9 09:00 sample.txt
-rwxr-xr-x 1 karamsettisaicharan karamsettisaicharan 16096 Sep 1 03:57 sjf
-rw-r--r-- 1 karamsettisaicharan karamsettisaicharan 518 Sep 1 04:01 sjf.c
```

ii)producer consumer communication using anonymous pipe

```
#include<stdio.h>
#include<unistd.h>
#include<string.h>
#include<fcntl.h>
int main(){
int fd[2];
pid_t pid;
char data[] = "Hello Producer";
char buffer[100];
pipe(fd);
pid = fork();
if(pid==0){
close(fd[1]);
read(fd[0],buffer,sizeof(buffer));
printf("Consumer received: %s\n",buffer);
close(fd[0]);
}
else{
close(fd[0]);
write(fd[1],data,strlen(data)+1);
printf("Producer sent: %s\n",data);
close(fd[1]);
}
return 0;
}
```

Output:

```
karamsettisaicharan@SaiCharan:~/Practicals$ nano practical5.c
karamsettisaicharan@SaiCharan:~/Practicals$ ./practical5
Producer sent: Hello Producer
Consumer received: Hello Producer
karamsettisaicharan@SaiCharan:~/Practicals$ |
```

6. practical6

```
#include<stdio.h>
#include<fcntl.h>
#include<unistd.h>
#include<sys/stat.h>
#include<string.h>
int main(){
int fd;
char message[100];
mkfifo("/tmp/myfifo",0666);
fd = open("/tmp/myfifo",O_RDONLY);
read(fd,message,sizeof(message));
printf("Server received the message: %s\n",message);
close(fd);
return 0;
}
```

```
#include<stdio.h>
#include<unistd.h>
#include<string.h>
#include<fcntl.h>
int main(){
int fd;
char message1[] = "Hello world";
fd = open("/tmp/myfifo",O_WRONLY);
write(fd,message1,strlen(message1)+1);
printf("Client sent the message: %s\n",message1);
close(fd);
return 0;
}
```

Output:

```
karamsettisaicharan@SaiCharan:~$ ./practical6_1
Client sent the message: Hello world
karamsettisaicharan@SaiCharan:~$ |
```

```
karamsettisaicharan@SaiCharan:~$ ./practical6
Server received the message: Hello world
```

ii) SIGNALS

```
GNU nano 2.7.1 Signal
#include <stdio.h>
#include <signal.h>
void handler(int sig)
{
    if(sig == SIGINT)
        printf("SIGINT received\n");
    if(sig == SIGTERM)
        printf("SIGTERM received\n");
    if(sig == SIGUSR1)
        printf("SIGUSR1 received\n");
}
int main()
{
    signal(SIGINT, handler);
    signal(SIGTERM, handler);
    signal(SIGUSR1, handler);
    printf("Signal handling program\n");
    raise(SIGINT);
    raise(SIGTERM);
    raise(SIGUSR1);
    return 0;
}
```

Output:

```
karamsettsaicharan@SaiCharan:~/Practicals$ nano signals.c
karamsettsaicharan@SaiCharan:~/Practicals$ ./signals
Signal handling program
SIGINT received
SIGTERM received
SIGUSR1 received
karamsettsaicharan@SaiCharan:~/Practicals$ |
```